Practical 3: State Management and Routing in React

<table>
<tr><td>Objective</td><td>To consume a REST API in React and handle asynchronous data with loading and error states.</td></tr>
<tr><td>Problem Statement</td><td>Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
• Use the Projects page built in Practical 2 as the integration point.
• Implement a loading spinner component shown while the request is in progress.
• Implement an error message component shown if the API call fails.
• Render at least the repository name and URL for each item returned.</td></tr>
<tr><td>Project Structure</td><td>Projects.jsx
├── useEffect() → triggers fetch on mount
├── useState: data, loading, error
├── [loading] → <Spinner />;
├── [error] → <ErrorMessage />;
└── [success] → <RepoList data={data} />;</td></tr>
<tr><td>Implementation</td><td>Project.jsx</td></tr>
</table>

```
import { useState, useEffect, useCallback } from 'react'

// You can change your GitHub username here to show your own
repositories
const GITHUB_USERNAME = 'Vaghasiya-Jemit-kanaiyalal'

function Spinner() {
  return (
    <div className="spinner-container">
      <div className="loading-spinner"></div>
      <p>Fetching repositories from GitHub...</p>
    </div>
  )
}

function ErrorMessage({ message, onRetry }) {
  return (
    <div className="repo-error-message">
      <p>⚠ Error: {message}</p>
      <button onClick={onRetry} className="retry-btn">
        Retry
      </button>
    </div>
  )
}

function RepoList({ data }) {
  return (
    <div className="projects-grid">
      {data.map((repo) => (
```

```
        <div key={repo.id} className="project-card">
          <div className="project-header">
            <h3>
              <a
                href={repo.html_url}
                target="_blank"
                rel="noopener noreferrer"
                className="project-title-link"
              >
                {repo.name.replace(/-/g, ' ').replace(/_/g, ' ')}
              </a>
            </h3>
          </div>

          <p className="project-desc">
            {repo.description || 'No description provided for this
repository.'}
          </p>

          <div className="project-meta">
            {repo.language && (
              <span className="tech-badge language-
badge">{repo.language}</span>
            )}
            <div className="repo-stats">
              <span title="Stars">⭐ {repo.stargazers_count}</span>
              <span title="Forks">🍴 {repo.forks_count}</span>
            </div>
          </div>

          <div className="project-footer">
            <a
              href={repo.html_url}
              target="_blank"
              rel="noopener noreferrer"
              className="view-repo-btn"
            >
              View on GitHub
            </a>
          </div>
        </div>
      ))}
    </div>
  )
}

function Projects() {
  const [data, setData] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [searchQuery, setSearchQuery] = useState('')
```

```
  const fetchRepos = useCallback(async () => {
    try {
      setLoading(true)
      setError(null)

      const response = await fetch(
        `https://api.github.com/users/${GITHUB_USERNAME}/repos?sort=u
pdated&per_page=100`
      )

      if (!response.ok) {
        if (response.status === 404) {
          throw new Error(`GitHub user "${GITHUB_USERNAME}" not
found.`)
        } else if (response.status === 403) {
          throw new Error('API rate limit exceeded. Please try again
later.')
        } else {
          throw new Error(`Failed to fetch repositories (Status
${response.status}).`)
        }
      }

      const repos = await response.json()
      setData(repos)
    } catch (err) {
      setError(err.message || 'An error occurred while fetching
repositories.')
    } finally {
      setLoading(false)
    }
  }, [])

  useEffect(() => {
    fetchRepos()
  }, [fetchRepos])

  const filteredData = data.filter((repo) => {
    const query = searchQuery.toLowerCase()
    const nameMatch = repo.name?.toLowerCase().includes(query)
    const descMatch = repo.description?.toLowerCase().includes(query)
    const langMatch = repo.language?.toLowerCase().includes(query)
    return nameMatch || descMatch || langMatch
  })

  return (
    <div className="projects-container">
      <h2>My Projects</h2>

      <div className="search-container">
```

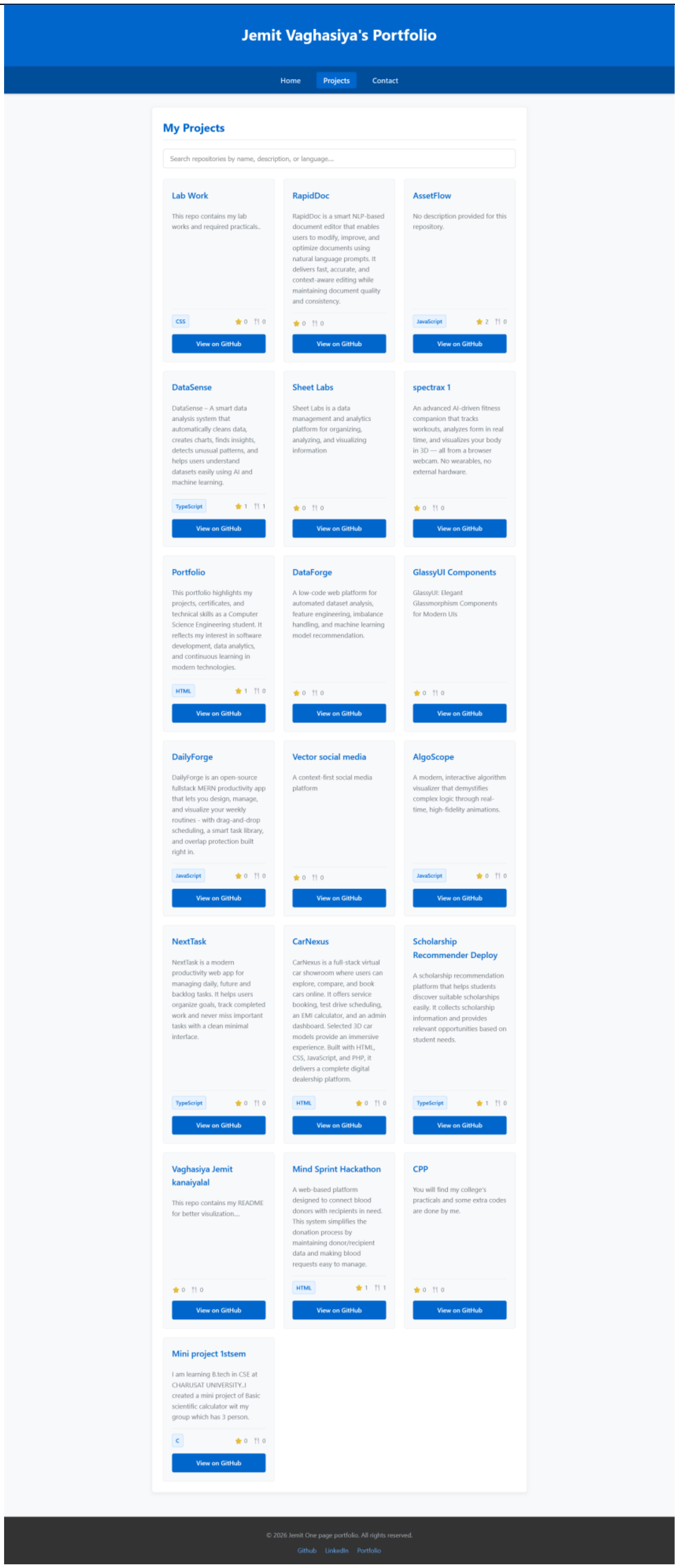
	(code below)

```
          <input
            type="text"
            placeholder="Search repositories by name, description, or
language..."
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
            className="search-input"
          />
          {searchQuery && (
            <button
              onClick={() => setSearchQuery('')}
              className="search-clear-btn"
              title="Clear search"
            >
              &times;
            </button>
          )}
        </div>

        {loading && <Spinner />}
        {!loading && error && <ErrorMessage message={error}
onRetry={fetchRepos} />}
        {!loading && !error && <RepoList data={filteredData} />}

        {!loading && !error && filteredData.length === 0 && (
          <div className="no-repos-found">
            <p>No repositories found matching "{searchQuery}".</p>
          </div>
        )}
      </div>
  )
}

export default Projects
```

Output	

Jemit Vaghasiya's Portfolio
Home
Projects
Contact
My Projects
Search repositories by name, description, or language...
Lab Work
This repo contains my lab works and required practicals..
CSS
View on GitHub
RapidDoc
RapidDoc is a smart NLP-based document editor that enables users to modify, improve, and optimize documents using natural language prompts. It delivers fast, accurate, and context-aware editing while maintaining document quality and consistency.
View on GitHub
AssetFlow
No description provided for this repository.
JavaScript
View on GitHub
DataSense
DataSense – A smart data analysis system that automatically cleans data, creates charts, finds insights, detects unusual patterns, and helps users understand datasets easily using AI and machine learning.
TypeScript
View on GitHub
Sheet Labs
Sheet Labs is a data management and analytics platform for organizing, analyzing, and visualizing information
View on GitHub
spectrax 1
An advanced AI-driven fitness companion that tracks workouts, analyzes form in real time, and visualizes your body in 3D — all from a browser webcam. No wearables, no external hardware.
View on GitHub
Portfolio
This portfolio highlights my projects, certificates, and technical skills as a Computer Science Engineering student. It reflects my interest in software development, data analytics, and continuous learning in modern technologies.
HTML
View on GitHub
DataForge
A low-code web platform for automated dataset analysis, feature engineering, imbalance handling, and machine learning model recommendation.
View on GitHub
GlassyUI Components
GlassyUI: Elegant Glassmorphism Components for Modern UIs
View on GitHub
DailyForge
DailyForge is an open-source fullstack MERN productivity app that lets you design, manage, and visualize your weekly routines - with drag-and-drop scheduling, a smart task library, and overlap protection built right in.
JavaScript
View on GitHub
Vector social media
A context-first social media platform
View on GitHub
AlgoScope
A modern, interactive algorithm visualizer that demystifies complex logic through real-time, high-fidelity animations.
JavaScript
View on GitHub
NextTask
NextTask is a modern productivity web app for managing daily, future and backlog tasks. It helps users organize goals, track completed work and never miss important tasks with a clean minimal interface.
TypeScript
View on GitHub
CarNexus
CarNexus is a full-stack virtual car showroom where users can explore, compare, and book cars online. It offers service booking, test drive scheduling, an EMI calculator, and an admin dashboard. Selected 3D car models provide an immersive experience. Built with HTML, CSS, JavaScript, and PHP, it delivers a complete digital dealership platform.
HTML
View on GitHub
Scholarship Recommender Deploy
A scholarship recommendation platform that helps students discover suitable scholarships easily. It collects scholarship information and provides relevant opportunities based on student needs.
TypeScript
View on GitHub
Vaghasiya Jemit kanaiyalal
This repo contains my README for better visulization....
View on GitHub
Mind Sprint Hackathon
A web-based platform designed to connect blood donors with recipients in need. This system simplifies the donation process by maintaining donor/recipient data and making blood requests easy to manage.
HTML
View on GitHub
CPP
You will find my college's practicals and some extra codes are done by me.
View on GitHub
Mini project 1stsem
I am learning B.tech in CSE at CHARUSAT UNIVERSITY.I created a mini project of Basic scientific calculator wit my group which has 3 person.
C
View on GitHub
© 2026 Jemit One page portfolio. All rights reserved.
Github
LinkedIn
Portfolio

Conclusion	In this practical, I connected a public REST API to the React project and displayed GitHub repositories. I also showed a loading spinner while data was loading and an error message if something went wrong. This helped us learn how to fetch and display data from an API in React.